

Pipeline di Web Scraping - Italia Semplice : Documentazione Tecnica

Autore: MIPA **Data:** 25 Marzo 2026

Versione: 1.0.0

1. Descrizione del Progetto

L'obiettivo di questa pipeline è l'estrazione sistematica e strutturata dei dati relativi agli interventi di semplificazione dal portale Italia Semplice. Si adotta un'architettura modulare che separa nettamente la logica di navigazione (motore), la configurazione dei parametri e le regole di estrazione (parsing). Questa scelta progettuale garantisce aggiornabilità: eventuali aggiornamenti futuri alla struttura del sito web richiedono modifiche mirate solo ai moduli di parsing, preservando l'integrità e la stabilità dell'intera automazione.

2. Struttura dei File

```
webscraping_italiasemplice/  
├─ main.py           # Coordina il flusso e il salvataggio dati  
├─ config.py         # Configurazioni generali  
├─ scraper_engine.py # Gestione del driver Selenium (Logica Anti-bot)  
├─ parser_utils.py   # Logica di estrazione (Regex e selettori XPATH)  
└─ requirements.txt  # Elenco delle dipendenze Python
```

3. Codice Sorgente

3.1 Configurazione (config.py)

```
import os  
  
# Scraping Config  
ID_START = 0  
ID_END = 700  
  
# Path Config  
BASE_DIR = os.path.dirname(os.path.abspath(__file__))  
OUTPUT_DIR = os.path.join(BASE_DIR, "output")  
DATA_DIR = os.path.join(OUTPUT_DIR, "data")  
LOG_DIR = os.path.join(OUTPUT_DIR, "logs")  
  
#Folders Creation  
os.makedirs(DATA_DIR, exist_ok=True) #output data folder  
os.makedirs(LOG_DIR, exist_ok=True) #log folder
```

```
#Output
CSV_OUTPUT = os.path.join(DATA_DIR, "webscraping_final.csv") #CSV for
incremental updates
LOG_FILE = os.path.join(LOG_DIR, "scraping_log.log") #Log file
```

3.2 Motore di Scraping (scraper_engine.py)

```
import undetected_chromedriver as uc

def get_options():
    options = uc.ChromeOptions()
    options.add_argument('--headless=new')
    options.add_argument('--no-sandbox')
    options.add_argument('--disable-gpu')
    options.add_argument('--window-size=1920,1080')
    return options

def get_driver():
    try:
        driver = uc.Chrome(options=get_options(), version_main=143)
    #specific version of Chrome, adjust as needed
    except Exception as e:
        driver = uc.Chrome(options=get_options())
    return driver
```

3.3 Main Entry Point (main.py)

```
import logging
import pandas as pd
import time
import os
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import config
from scraper_engine import get_driver
from parser_utils import extract_label, extract_procedure_name,
parse_interventions

# Setup Logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.FileHandler(config.LOG_FILE),
        logging.StreamHandler()
    ]
)
```

```

)

def run_pipeline():
    logging.info("Avvio scraping")
    driver = None

    for current_id in range(config.ID_START, config.ID_END + 1):
        url =
f"https://www.italiasemplice.gov.it/ricerca/dettaglio/{current_id}"
        logging.info(f"Elaborazione ID {current_id}: {url}")

        try:
            if driver is None: driver = get_driver()
            driver.get(url)
            try:
                WebDriverWait(driver,
5).until(EC.presence_of_element_located((By.XPATH, "//*[contains(text(),
'Settore:')]"))))
            except:
                logging.warning(f"ID {current_id}: Pagina vuota o non
trovata")
                continue

            #data extraction
            body_text = driver.find_element(By.TAG_NAME, "body").text
            nome_proc = extract_procedure_name(driver)
            settore = extract_label(body_text, "Settore")
            categoria = extract_label(body_text, "Categoria")
            beneficiario = extract_label(body_text, "Beneficiario")
            tipo_pa = extract_label(body_text, "Tipo PA Responsabile")
            interventions = parse_interventions(driver) #interventi

            rows = [] #list of dicts to store data for each intervention
            for inter in interventions:
                row = {
                    "ID": current_id,
                    "Nome Procedura": nome_proc,
                    "Settore": settore,
                    "Categoria": categoria,
                    "Beneficiario": beneficiario,
                    "Tipo PA Responsabile": tipo_pa,
                    **inter,
                    "URL": url
                }
                rows.append(row)

            #Incremental CSV writing/saving
            df_temp = pd.DataFrame(rows)
            file_exists = os.path.isfile(config.CSV_OUTPUT)
            df_temp.to_csv(config.CSV_OUTPUT, mode='a', index=False,
header=not file_exists, sep=';', encoding='utf-8-sig')

            logging.info(f"ID {current_id} scaricato con successo
({len(rows)} interventi salvati)")

```

```

        if current_id % 20 == 0: #memory management: restart driver
every 20 IDs
            driver.quit()
            driver = None

    except Exception as e:
        logging.error(f"Errore critico su ID {current_id}: {e}")
        if driver:
            driver.quit()
            driver = None

    if driver: driver.quit()
    logging.info("Scraping completato")

if __name__ == "__main__":
    run_pipeline()

```

3.4 Utility e Funzioni (parser_utils.py)

```

import re
from selenium.webdriver.common.by import By

def extract_label(body_text, label):
    m = re.search(rf"{label}:\s*(.*)", body_text, re.IGNORECASE)
    if m:
        return m.group(1).split('\n')[0].strip()
    return ""

def extract_procedure_name(driver):
    nome_proc = "N/D"
    try:
        el = driver.find_element(By.XPATH, "//div[contains(@class, 'card-body')]/h3 | //div[contains(@class, 'card-body')]/h5 | //div[contains(@class, 'card-header')]/h5")
        nome_proc = el.text.strip()
    except:
        try:
            full_text = driver.find_element(By.TAG_NAME, "body").text
            parts = full_text.split("Dettaglio Procedura")
            [1].split("Settore:")[0].split('\n')
            valid_lines = [p.strip() for p in parts if p.strip() and "Scarica" not in p]
            if valid_lines: nome_proc = valid_lines[0]
        except: pass
    return nome_proc.replace("Scarica XLSX/CSV", "").strip()

def parse_interventions(driver):
    interventions_list = []
    cards = driver.find_elements(By.XPATH, "//div[contains(@class, 'card')]

```

```

and .//*[contains(text(), 'Tipologia di intervento')]]")

    if not cards:
        return [{"Intervento": "", "Anno": "", "Descrizione Intervento":
"", "Tipo Intervento": "", "Natura Intervento": "", "Riferimenti": ""}]

    for card in cards:
        # Inizializzazione sicura variabili
        tipo = natura = riferimenti = anno = title_clean = ""
        desc = []

        try:
            card_text = card.text
            lines = [l.strip() for l in card_text.split('\n') if l.strip()]
            def get_val(keyword):
                for i, line in enumerate(lines):
                    if keyword.lower() in line.lower() and i + 1 <
len(lines):
                        res = lines[i+1]
                        if res.lower() == keyword.lower() and i + 2 <
len(lines): #to avoid cases where value is missing and keyword is repeated
                            return lines[i+2]
                        return res
                return ""
            tipo = get_val("Tipologia di intervento")
            natura = get_val("Natura intervento")
            riferimenti = get_val("Riferimenti")

            try:
                title_el = card.find_element(By.TAG_NAME, "h5")
                title_full = title_el.text.strip()
            except:
                title_full = lines[0] if lines else ""
            anno_m = re.search(r'\((\d{4})\)', title_full)
            if anno_m:
                anno = anno_m.group(1)
                title_clean = title_full.replace(f"({anno})", "").strip()
            else:
                title_clean = title_full
            start_capture = False
            for line in lines:
                if line == title_full:
                    start_capture = True; continue
                if any(k in line for k in ["Tipologia di intervento",
"Natura intervento", "Riferimenti"]):
                    break
                if start_capture: desc.append(line)

            interventions_list.append({
                "Intervento": title_clean, "Anno": anno,
                "Descrizione Intervento": " ".join(desc).strip(),
                "Tipo Intervento": tipo, "Natura Intervento": natura,
                "Riferimenti": riferimenti
            })

```

```
        except:
            interventions_list.append({"Intervento": "Errore parsing"})

    return interventions_list
```

4. Installazione e Dipendenze

Per eseguire il progetto, installare le dipendenze elencate in `requirements.txt`:

```
pip install -r requirements.txt
```